Step 1 - Check your workspace and baseline

1. Open VS Code using the shortcut on the Desktop.
2. Open the lab's workspace folder by clicking File > Open Folder and opening the C:\QASAIEFGC folder. The workspace root should contain `src`, `tests`, `scenario`, `templates` and `lab_outputs`. If access is missing, report **ACCESS** to the facilitator using the session's communication route.
3. Open `scenario/change-request.md` and `scenario/team-constraints.md`. CHG 104 permits changes only to `src/priority.py`: normalise valid priorities and reject every invalid value. Keep the public function and the supplied tests unchanged. Use only the supplied fictional data; no remote repository, push, merge or deployment is required.
4. Open the integrated terminal at the workspace root and run:

   python --version

   python -m unittest discover -s tests -v > lab_outputs/runs/CHG104-attempt01-baseline.txt 2>&1

5. Open `lab_outputs/runs/CHG104-attempt01-baseline.txt` to inspect the actual result. Keep this file unchanged throughout the lab. The starter intentionally fails acceptance checks.
6. In `lab_outputs/01-workflow-record.md`, record the run time, visible Python/VS Code/Copilot versions, selected model, agent environment and permitted tools or confirmation settings. Add the Python version beside the other versions. Record a visible revision or name your starting snapshot. If a value is unavailable, say so rather than guessing.

Step 2 - Configure a reusable workflow

Separate stable repository rules from a reusable procedure for one reviewable change.

1. In the prepared workspace, create `.github/copilot-instructions.md` using `templates/copilot-instructions.template.md` as a scaffold. Replace the bracketed prompts with concise rules for repository context, source-of-truth files, permitted edits, tests and human review. Keep CHG 104's acceptance details in its request file.
2. Create `.github/skills/reviewable-change/SKILL.md` using `templates/skill.template.md`. Retain the skill name `reviewable-change` and write a specific description. Complete its required inputs, readiness check, ordered procedure, evidence outputs and stopping conditions. The YAML header begins on the first line and ends before the Markdown body. Remove scaffold text.
3. Make the procedure require a proposed plan before editing, a human decision on the plan, actual verification evidence and clarification when acceptance criteria are missing. Keep instructions distinct from tool permissions: text instructions guide behaviour but do not enforce a security boundary.
4. Check the skills available in your Copilot chat environment. If the deployed interface exposes `/reviewable-change`, select the recognised skill. Record what the interface actually shows and how you invoked or discovered the skill in `lab_outputs/01-workflow-record.md`. Save supporting observations in `lab_outputs/runs/skill-discovery.txt`.

Step 3 - Execute and review CHG 104

Use the configured workflow for a scoped patch and compare model claims with your own review and executed tests.

1. Start a fresh Copilot conversation and invoke your `reviewable-change` skill. Provide the following workspace files as context using the controls available in your environment: `scenario/change-request.md`, `scenario/team-constraints.md`, `src/priority.py` and `tests/test_priority.py`. Check permitted tools before allowing operations.
2. Send this prompt:

   Use reviewable-change for CHG 104. Read scenario/change-request.md,

   scenario/team-constraints.md, src/priority.py and tests/test_priority.py.

   First give a short plan and identify any missing requirement. Do not edit yet.

3. Save the actual prompt and response to `lab_outputs/runs/CHG104-attempt01-chat.md`. Review the proposed plan against the request. Correct invented requirements, changed tests or extra files before proceeding. When the plan is suitable, send:

   Implement the reviewed CHG 104 plan within the supplied scope.

   Show the changed files and explain the patch. If you cannot execute a

   check, state that clearly. Do not merge or deploy anything.

4. Add the response to the same chat record. Inspect the actual diff or the editor's before/after comparison. The code patch may change only `src/priority.py`; your separately authored workflow and evidence files are expected lab outputs. Confirm that the public function is unchanged, all four valid priorities remain valid, case and surrounding whitespace are normalised, and all invalid values raise `ValueError`. Reject out-of-scope changes.
5. Run the unchanged tests yourself:

```
python -m unittest discover -s tests -v > lab_outputs/runs/CHG104-attempt01-after.txt 2>&1
```

6. Open the result file. Complete the CHG 104 section of `lab_outputs/01-workflow-record.md`: exact prompt/context, accepted or revised plan, changed files, test command and log, actual outcome, human diff review and an uncovered risk.

Step 4 - Trial an incomplete request

Test the workflow's readiness check when a request lacks an essential business rule, and record your own decision about the response.

1. Preserve your CHG 104 files and results. Read `scenario/incomplete-request.md` and `scenario/team-constraints.md` in the prepared workspace.
2. Start a fresh Copilot conversation. Invoke the same `reviewable-change` skill and provide those two files as context. Send:

   Use reviewable-change to assess CHG 105 in scenario/incomplete-request.md.

   Apply scenario/team-constraints.md. Check readiness before proposing a

   Change. Identify missing acceptance criteria and the owner who must

   resolve them. Do not edit any file or invent a priority mapping.

3. Save the exact prompt and actual response in `lab_outputs/runs/CHG105-attempt01-chat.md`.
4. Complete the CHG 105 section of `lab_outputs/01-workflow-record.md`. Separate expected behaviour from observed behaviour, explain any difference, and name the human decision and next owner.

Step 5 - Check your workflow evidence

Make the first activity's result reviewable before using it in a team operating model.

1. Finish `lab_outputs/01-workflow-record.md`, including the reflection on shared context versus change-specific input.
2. Check these outputs in your prepared workspace:

Output	Check
`.github/copilot-instructions.md`	Specific repository scope, verification and review rules
`.github/skills/reviewable-change/SKILL.md`	Complete reusable procedure and readiness check
`src/priority.py`	Reviewed CHG 104 patch within scope
`lab_outputs/01-workflow-record.md`	Environment, discovery, both trials and human decisions recorded
`lab_outputs/runs/`	Baseline, after-change tests and actual prompts/responses preserved

3. Append a checkpoint status to the workflow record: **READY** if the evidence is complete, **RETRY** if a correction remains, or **ACCESS** if environment access prevented execution. Include one sentence identifying your weakest evidence.

Step 6 - Define pilot scope and ownership

Turn one reusable workflow into a bounded pilot with explicit responsibilities.

1. Read `scenario/pilot-evidence.md` and revisit `scenario/team-constraints.md`. All figures and comments are fictional training inputs, not measured Copilot effects.
2. In `lab_outputs/02-operating-model.md`, complete **Goal and pilot boundary**. Name an engineering outcome, the problem it addresses, eligible tasks, exclusions, cohort and four-week period. Keep the scope small enough for a team to review.
3. Complete **Ownership and routine**. Assign accountable roles for maintaining the shared instructions/skill, owning the pilot decision, reviewing code, resolving business ambiguity and approving changes to data or tool access. A role is sufficient in this fictional scenario; you do not need real colleagues' names.
4. Specify how a developer is onboarded, where they get support, what the weekly review checks, how a pattern change is reviewed and versioned, how adopters receive the version, and how a faulty version is rolled back. These are your proposed operating practices, not actions you have performed.

Step 7 - Calculate and interpret pilot measures

Read the fictional pilot evidence without confusing adoption, workflow quality, delivery outcomes or risk.

1. Use `scenario/pilot-evidence.md` and complete **Measures** in `lab_outputs/02-operating-model.md`.
2. Calculate evidence completeness for both periods: complete review evidence divided by eligible changes. Show the fraction and percentage for each period.
3. Calculate the reopened-change rate for both periods: changes reopened within seven days divided by eligible changes. Show the fraction and percentage for each period. Compare the change in percentage points.
4. Calculate relative change in median approval time: `(pilot − before) / before × 100`. Include the original values and their units. Keep a relative percentage change distinct from a change in percentage points.
5. Interpret active users as a usage signal. Interpret the survey using respondents as its denominator, and identify the missing responses. Distinguish these from engineering outcomes.
6. Record the severe-incident status and at least two limitations of the comparison. Consider sample size, changing task mix or staffing, and survey non-response.

Step 8 - Make a bounded scale decision

Use the evidence and ownership model to recommend a decision that another person can act on and later reassess.

1. Review your calculations in `lab_outputs/02-operating-model.md` against the gates in `scenario/pilot-evidence.md`: at least 85% complete evidence, no increase in reopened-change rate, zero unresolved severe incidents, and a named maintainer plus support route. These are fictional scenario choices, not universal thresholds.
2. Complete the worksheet's **Decision** section. Choose **expand**, **adapt** or **pause**, and state the scope affected. Distinguish continuing a contained pilot from expanding to other squads.
3. Support the decision with the actual scenario evidence. Explicitly address the unresolved severe incident and the reopened-change rate, alongside any favourable measures.
4. Name an immediate action and accountable owner, the evidence still needed, a testable re-entry condition, and a review date relative to the pilot (for example, “within five working days of this review”). If any limited work continues, state who can authorise it and under what containment conditions.

Step 9 - Review, revise and complete your evidence

1. Challenge a specific claim, route an exception to its owner and leave a reasoned revision supported by evidence.
2. Read your proposed decision and its evidence against that exception.
3. Check contrary evidence, identify the next accountable owner and state a correction or re-entry condition. Record the discussion or self-review in `lab_outputs/03-scale-review.md`.
4. Revise the relevant record and identify the exact change in the scale-review record. In its workplace-action field, name a task, pilot owner, one measure and a target date. Check the completion criteria below and record any unmet criterion openly.